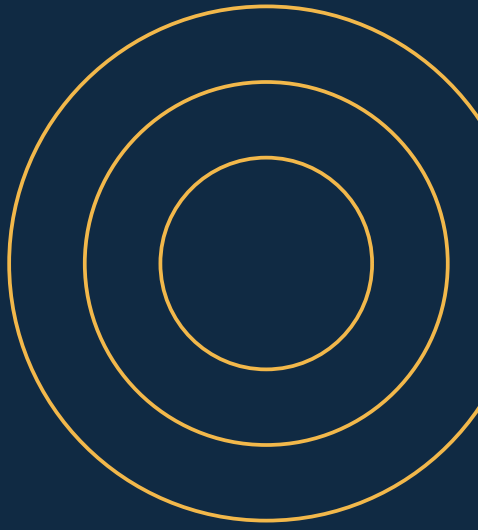
EduGenie

Google Gemini Powered Learning Assistant

Project Report

A generative AI study companion: explain, summarize, quiz and plan



Stack: Python, Flask, SQLite, google-genai SDK, HTML/CSS/JavaScript

Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives and Scope
5. Existing Systems and Proposed System
6. Requirements
7. System Architecture
8. Module Description
9. Technology Stack
10. Database Design
11. API Design
12. Prompt Engineering
13. Safety, Privacy and Academic Integrity
14. Testing and Evaluation
15. Limitations
16. Future Scope
17. Conclusion
18. References

1. Abstract

EduGenie is a web-based learning assistant that uses Google Gemini models to help students study. A student can ask questions and receive explanations pitched at their level, upload class notes and get summaries, generate practice quizzes and flashcards from those notes, and build a study plan around an exam date. The assistant answers in a structured, teaching-oriented way instead of only giving final answers.

The system has a Flask back end, a lightweight HTML, CSS and JavaScript front end, and a SQLite database for chat history and quiz results. All model calls are isolated in one tutor service, which applies prompt templates, safety rules and output validation. This report describes the problem, the design, the modules, the data model, the prompts and the plan for testing and evaluation.

2. Introduction

Students today study from textbooks, slides, videos and web pages, but personal feedback is scarce. A teacher cannot answer every follow-up question at the moment it arises, and search results rarely match a learner's level. Large language models can fill part of this gap by explaining a concept several ways, checking understanding through questions, and working directly from the student's own material.

Google Gemini models accept long text and documents as input and can return structured (JSON) output, which suits an educational tool: the same model can chat, summarize a chapter, and produce a machine-readable quiz. EduGenie builds a focused study product on those abilities.

3. Problem Statement

Students lack an always-available, personalized helper that works from their own notes, adapts to their level, and turns passive reading into active practice. General chatbots answer questions but do not keep study context, do not test the learner, and can encourage copying answers instead of learning. The goal is to design a learning assistant that explains, tests and plans, while keeping the student's own understanding at the center.

4. Objectives and Scope

- Explain topics at a chosen level (beginner, intermediate, advanced) with examples and analogies.
- Summarize uploaded notes into key points and definitions.
- Generate multiple-choice quizzes and flashcards from notes or a topic, and score the student's answers.
- Create a day-by-day study plan from a subject list and an exam date.
- Keep chat and quiz history so progress can be reviewed.
- Refuse to be a shortcut for graded work: guide with hints when the student asks for a full solution.

In scope: text chat, text and PDF notes, quizzes, flashcards, study plans, history. **Out of scope for this version:** voice input, image or handwriting recognition, teacher dashboards and user accounts with passwords.

5. Existing Systems and Proposed System

Aspect	Typical existing tools	EduGenie
Source of answers	General web knowledge	General knowledge plus the student's own uploaded notes
Level of explanation	One style for everyone	Chosen level, with simpler rewording on request
Active practice	Rarely included	Quizzes and flashcards generated on demand
Planning	Separate apps	Study plan built into the same workspace
Learning behavior	Gives the final answer	Hints first, full steps on request for practice problems
Progress	None or manual	Stored chat and quiz history

6. Requirements

6.1 Functional requirements

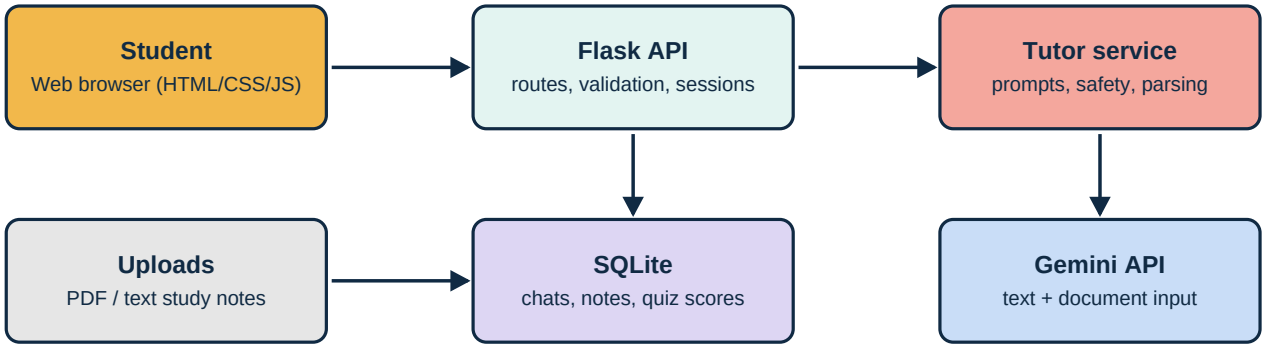
ID	Requirement
F1	The student can start a chat and ask questions on any study topic.
F2	The student can set a level and a subject for the session.
F3	The student can upload a .txt or .pdf file and request a summary.
F4	The system generates a quiz (5 to 15 questions) with correct answers and short explanations.
F5	The system generates flashcards as term and definition pairs.
F6	The system generates a study plan from subjects, exam date and daily hours.
F7	The system stores chats, quiz attempts and scores and shows history.

6.2 Non-functional requirements

- **Usability:** works on desktop and phone screens with no installation for the student.
- **Reliability:** clear messages on rate limits, invalid keys, blocked responses and network problems.
- **Security:** the API key stays on the server in an environment file; uploads are size-limited and type-checked.
- **Performance:** chat replies stream or return in a few seconds; long summaries show a progress state.
- **Maintainability:** all Gemini logic in one module; model names set in configuration.

7. System Architecture

EduGenie uses a three-layer design. The browser handles input and display. The Flask API validates requests and stores data. The tutor service is the only component that calls Gemini, so prompts, safety rules and response parsing can be changed or tested in one place.



Requests flow from the student to the Flask API; only the tutor service talks to Gemini.

8. Module Description

Module	Responsibility
Chat tutor	Answers questions using the session's subject and level, keeps the last several turns as context, and asks a follow-up question to check understanding.
Notes summarizer	Extracts text from an uploaded file, splits long text into sections if needed, and returns key points, definitions and a short recap.
Quiz generator	Requests multiple-choice questions as JSON (question, four options, answer index, explanation), validates the shape, and scores the student's answers.
Flashcard generator	Produces term and definition pairs from notes or a topic and shows them as flip cards.
Study planner	Turns subjects, exam date and daily hours into a dated plan with topics, revision days and a final review.
History	Lists earlier chats and quiz scores from the database so progress can be reviewed.

9. Technology Stack

Layer	Technology	Reason
Language	Python 3.10+	Strong AI library support, easy to run in VS Code.
Back end	Flask	Small and clear for a handful of JSON endpoints.
AI	Google Gemini via google-genai SDK	Long-context text and document input, JSON output.
Database	SQLite	Zero setup, file-based, enough for a single-user or small deployment.
File reading	pypdf	Extracts text from PDF notes.
Front end	HTML, CSS, JavaScript	No build step; runs directly from the Flask server.
Config	python-dotenv	Keeps the API key and model names out of the code.
Testing	pytest	Fast tests with the Gemini calls replaced by stand-ins.

10. Database Design

SQLite holds four tables. Foreign keys link messages and quiz attempts to their session.

Table	Main columns
sessions	id, title, subject, level, created_at
messages	id, session_id, role (student or tutor), content, created_at
notes	id, session_id, filename, extracted_text, summary, created_at
quiz_attempts	id, session_id, topic, questions_json, answers_json, score, total, created_at

11. API Design

Endpoint	Method	Purpose
/api/chat	POST	Send a message with session id, subject and level; returns the tutor reply.
/api/notes	POST	Upload a .txt or .pdf file; returns extracted text length and a summary.
/api/quiz	POST	Create a quiz from a topic or a saved note; returns validated questions.
/api/quiz/score	POST	Submit answers; returns score and per-question explanations.
/api/flashcards	POST	Create flashcards from a topic or a saved note.
/api/plan	POST	Create a study plan from subjects, exam date and daily hours.
/api/history	GET	List sessions and quiz scores.
/api/health	GET	Report whether the API key and models are configured.

12. Prompt Engineering

Each feature has its own prompt template. Four rules are shared across them.

- **Role and level.** A system instruction sets the tutor role, the student's level and subject, and asks for short paragraphs, examples and a closing check question.
- **Grounding in notes.** When notes are supplied, the prompt tells the model to answer from them first and to say clearly when the notes do not cover something.
- **Structured output.** Quiz, flashcard and plan requests ask for one exact JSON shape; the service strips code fences, parses, and validates every field before use.
- **Teaching behavior.** For homework-style problems the tutor gives a hint or the first step, then full working only if the student asks or has tried.

Example instruction (simplified):

```
You are EduGenie, a patient tutor for a {level} student studying {subject}.
Explain step by step with one concrete example. Keep answers under 200 words.
If notes are provided, use them and say when they do not cover the question.
End with one short question that checks understanding.
```

13. Safety, Privacy and Academic Integrity

- **Accuracy.** Models can be wrong. The interface reminds students to check important facts against their textbook or teacher.
- **Notes over guesses.** Answers grounded in uploaded notes are labeled, and gaps are stated instead of filled with invented details.
- **Integrity.** The tutor prefers hints and worked examples over complete answers to graded-looking tasks.
- **Content safety.** Gemini's built-in safety filters stay on; blocked responses show a clear message and a suggestion to rephrase.
- **Privacy.** Notes and chats are stored only in the local database; the API key never reaches the browser; no personal data beyond a session title is required.
- **Upload safety.** Only .txt and .pdf files, with a size limit, and text is extracted without executing anything.

14. Testing and Evaluation

Automated tests replace Gemini calls with stand-ins so they run without an API key. Planned checks: request validation for every endpoint, quiz JSON validation and rejection of malformed output, quiz scoring, file type and size limits, database reads and writes, and error responses when Gemini fails.

Manual test cases:

Test	Expected result
Ask a question at beginner and advanced levels	The advanced answer uses more precise terms and less hand-holding.
Upload notes, then ask something they do not cover	The tutor says the notes do not cover it before adding general knowledge.
Generate a 10-question quiz	Ten valid questions, four options each, one correct answer marked.
Submit quiz answers	Correct score and an explanation for each wrong answer.
Set an exam date 14 days ahead	Plan spans 14 days and ends with a review day.
Start with no API key	Warning appears and requests fail with a clear message.

Evaluation measures for a pilot with students: quiz score improvement between first and second attempts, time saved preparing revision material, and a short usefulness rating for each feature. This report presents the design and test plan; measured results would be added after a pilot.

15. Limitations

- Generated explanations and quiz answers can contain mistakes and need human review.
- PDF text extraction fails on scanned pages or complex layouts.
- Very long documents may need splitting, which can lose cross-chapter links.
- The first version has no login, so history is shared by everyone using the same installation.
- Free-tier API limits can slow heavy use.

16. Future Scope

- User accounts, teacher dashboards and class-level progress reports.
- Voice questions and spoken explanations.
- Reading diagrams, photos of handwritten notes and textbook pages with Gemini's image understanding.
- Spaced-repetition scheduling for flashcards.
- Multiple languages for explanations and quizzes.
- Source highlighting that shows which part of the notes supported each answer.

17. Conclusion

EduGenie shows how a single Gemini-powered service can support the full study cycle: understanding a topic, condensing notes, practicing with quizzes and flashcards, and planning revision. Keeping all model logic in one service, validating structured output, grounding answers in the student's notes and favoring hints over final answers make the design practical and responsible. The modular Flask structure also leaves room for accounts, voice and image features in later versions.

18. References

- Google AI for Developers, Gemini API documentation: https://ai.google.dev

- Flask documentation: https://flask.palletsprojects.com
- Python SQLite3 module documentation: https://docs.python.org/3/library/sqlite3.html
- pypdf documentation: https://pypdf.readthedocs.io